

Práctico 4: Dinic.

1) networks, hallar un flujo maximal y un corte minimal usando el algoritmo de Dinic. Si tiene elección use el orden alfabético $S < A < B$. Calcular el valor del flujo maximal obtenido y la capacidad del corte minimal obtenido.

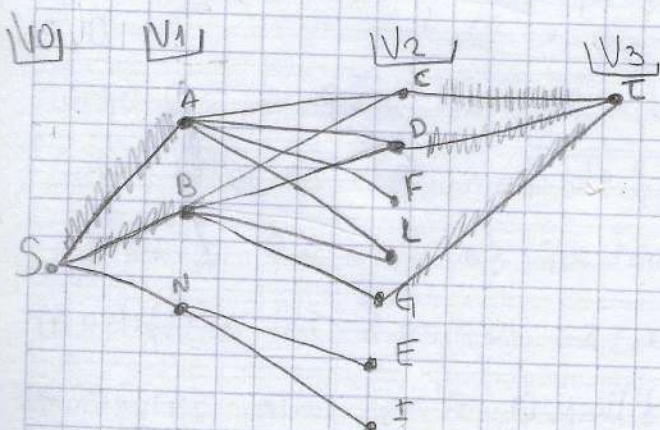
2) 2NA + el que no llega a T) 4 caminos en el primero 9 en el segundo $V(f_{max}) = 377$

SA: 144 640	CT: 860	HT: 2050
SB: 96 800	CJ: 5	HM: 40
SN: 150 150	DT: 86 160	ID: 22 180
AC: 100 2035	EC: 100 8520	IG: 160
AD: 70 811	ED: 15 180	MT: 30
AF: 85 3050	EG: 100	NE: 110
AL: 17 1230	FH: 180	NI: 40 08
BC: 10	FK: 100 3520	KT: 120 66
BD: 17 16	GH: 305	UH: 305
BG: 102 2232	LK: 60 5845	
BL: 35 25	GT: 80	

Dinic $V_i = \{X: dp(S, X) = i\}$
 $v_i \rightarrow v_{i+1}$

Bloqueante si al correr Greedy, el flujo no se puede aumentar.

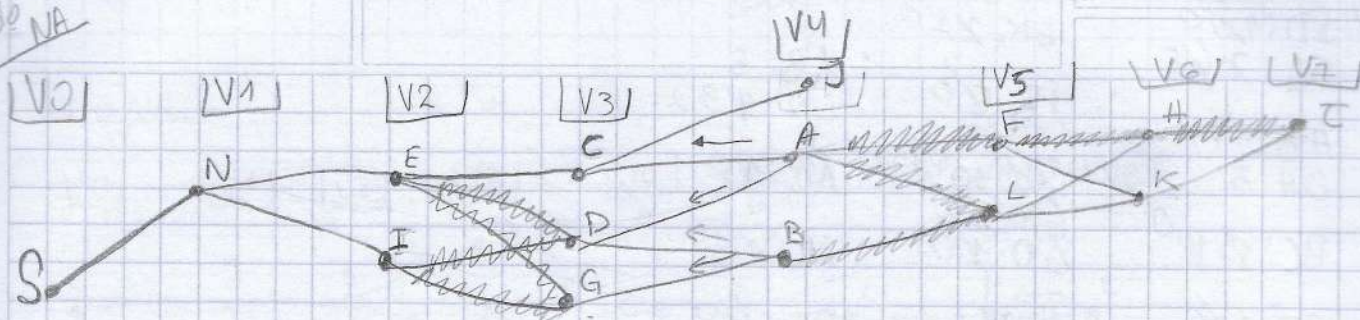
- 1) $P=0$
- 2) Construir NA a partir de f
- 3) hallar flujo bloqueante g en el NA
- 4) Usar g para cambiar f
- 5) Repito 2) hasta que pueda llegar a T.



Verm. 005	
SACT:	80
SADT:	64
SBDT:	16
SBGT:	80

235
-18
217

2da NA



← SNECAFHT: 15

← SNECAFKT: 65

← SNECAFKT: 5

← SNEBALHT: 5

← SNEBALKT: 5

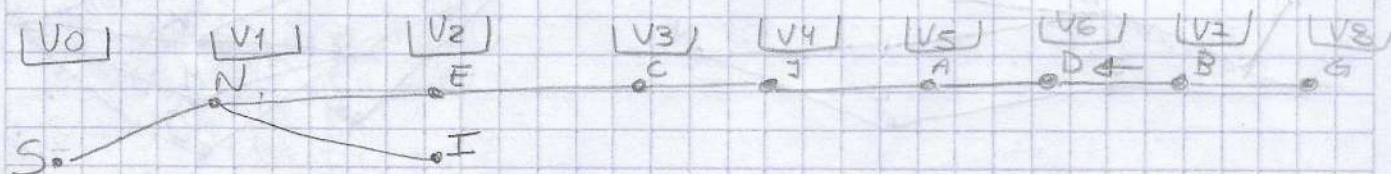
← SNEGBLKT: 10

← SNIDALKT: 1

← SNISBLKT: 15

← SNIGBLKT: 10

3er NA



No puedo llegar a C, mi algoritmo termina.

Corte $S = \{ \check{S}, \check{N}, \check{E}, \check{I}, \check{J}, \check{A}, \check{D}, \check{B}, \check{G} \}$

Check: $v(f) = out(f) = 377$

Check con S : $cap(S) = cap(S, \bar{S}) = \sum [x \in S] [y \notin S] [xy \in E] c(xy)$

$= C(\check{C}) + C(\check{A}) + C(\check{D}) + C(\check{B}) + C(\check{G})$

$= 80 + 85 + 17 + 80 + 35 + 80$

$= 377$

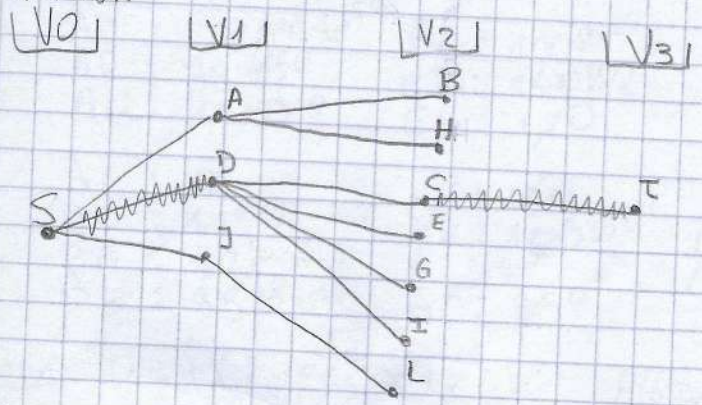
Como corte es minimal y f un flujo maximal y tengo $cap(S) = 377 = v(f)$

$\Rightarrow$ fin check.

b) SA: 15/0
 SD: 200
 SJ: 7/5
 AB: 17/2
 AH: 5
 BC: 15/0
 CT: 200
 DC: 26/11/23
 DE: 5/1
 DG: 10/0
 DI: 0/4
 EF: 5/0
 EK: 2/0
 FI: 5/0
 GK: 10/2
 GH: 3/2
 GO: 1/0
 HN: 4
 IF: 4/2
 JL: 7/5
 KC: 10/0
 LB: 5
 LN: 4/2
 MT: 1/0
 NC: 6/4
 OT: 1/9

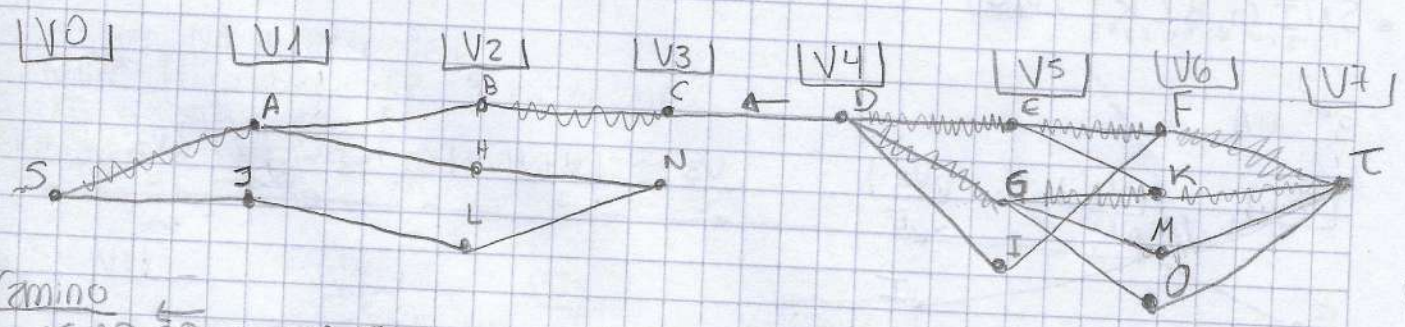
3NA 1-2-2
 $\sqrt{f_{max}} = 32$

Primer NA



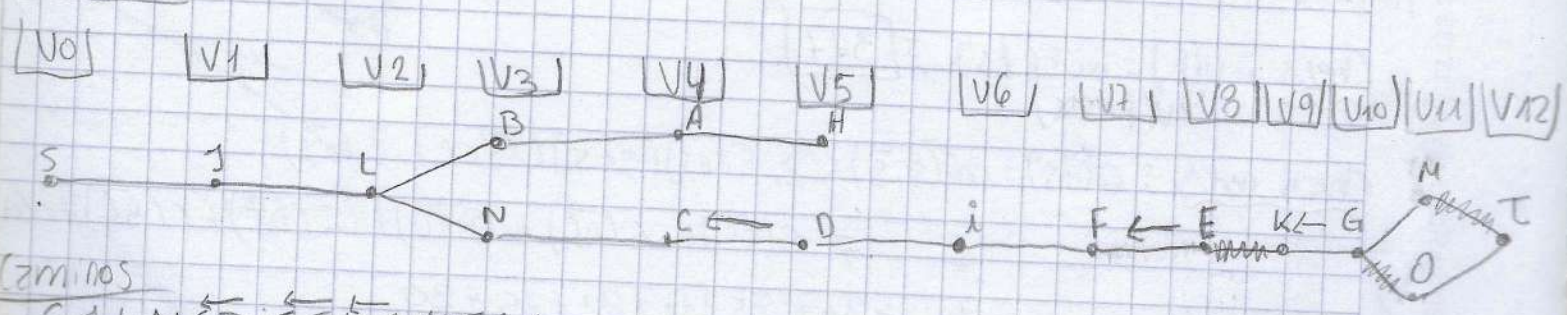
camino
 SDCT: 20

Segundo NA



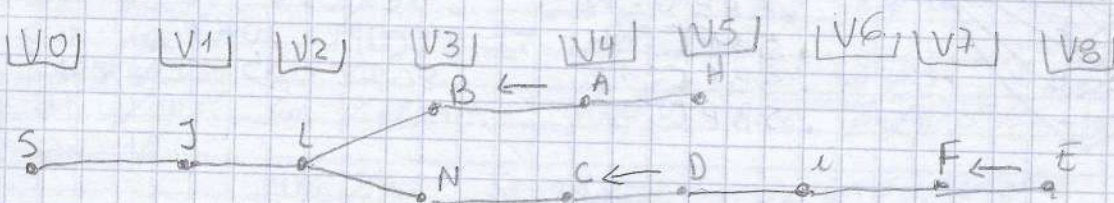
camino
 • SAB CDEFT: 5
 • SAB C D G K T: 10

Tercer NA



caminos
 • SJ L N C D I F E K G M T: 1
 • SJ L N C D I F E K G O T: 1

Cuanto NA



No puedo llegar a t, el algoritmo termina.

Corte minimal $S = \{S, J, L, B, A, H, N, C, D, I, F, E\}$

$$\begin{aligned} \text{Check: } \text{cap}(S) &= \text{cap}(S, \bar{S}) = \sum [x \in S][y \in \bar{S}][\vec{xy} \in E] c(\vec{xy}) \\ &= c(\vec{CL}) + c(\vec{DG}) + c(\vec{FT}) + c(\vec{EK}) \\ &= 20 + 10 + 5 + 2 \end{aligned}$$

$$\text{cap}(S) = \boxed{37}$$

$$V(f_{\max}) = \text{out}_f(s) = 20 + 5 + 10 + 1 + 1 = \boxed{37}$$

Luego como mi corte es minimal y f un flujo maximal y además $\text{cap}(S) = 37 = V(f_{\max})$

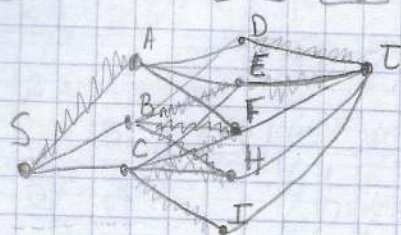
Termino check, esta todo ok.

c) SA: 20 110 45
 SB: 69 68 50
 SC: 148 140 110
 AD: 145 712
 AE: 19 819
 AF: 18 780
 BD: 930
 BE: 40
 BF: 140
 BH: 40
 CE: 190 189
 CF: 40
 CH: 200
 CI: 295
 DI: 1
 DJ: 777
 EI: 16
 EJ: 750
 FI: 146 132 118 117 115 110
 GI: 5
 HI: 28 144
 IT: 180
 JT: 720

pel de fibra de caña de azúcar.

1^{ra} NA 9-3-1-2 $v(fmax) = 108$.

V0 V1 V2 V3



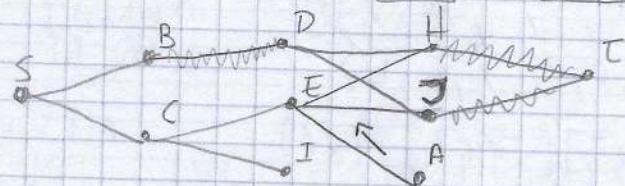
Caminos

- SADT: 9
- SAET: 11
- SBET: 4
- SBFT: 14
- SBHT: 1

- SCET: 1
- SCFT: 4
- SCHT: 20
- SCIT: 15

2da NA

V0 V1 V2 V3 V4

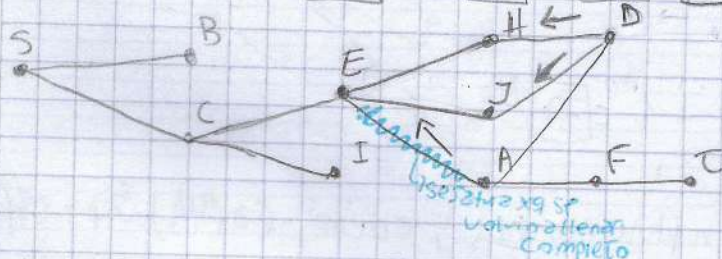


Caminos

- SBDHT: 4
- SBDJT: 5
- SCEJT: 2

3er NA

V0 V1 V2 V3 V4 V5



Caminos

- SCEAFT: 11

4to NA

V0 V1 V2 V3 V4 V5 V6 V7

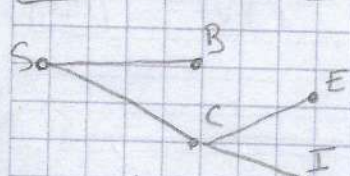


Caminos

- SCEHDAFT: 2
- SCEJDAFT: 5

5to NA

V0 V1 V2



No puedo llegar a T
el algoritmo termina.

$$v(fmax) = 5 + 2 + 11 + 2 + 5 + 4 + 15 + 20 + 4 + 1 + 1 + 14 + 4 + 11 + 9 = 108$$

Corte mn. mal: $S = \{S, B, C, E, I\}$

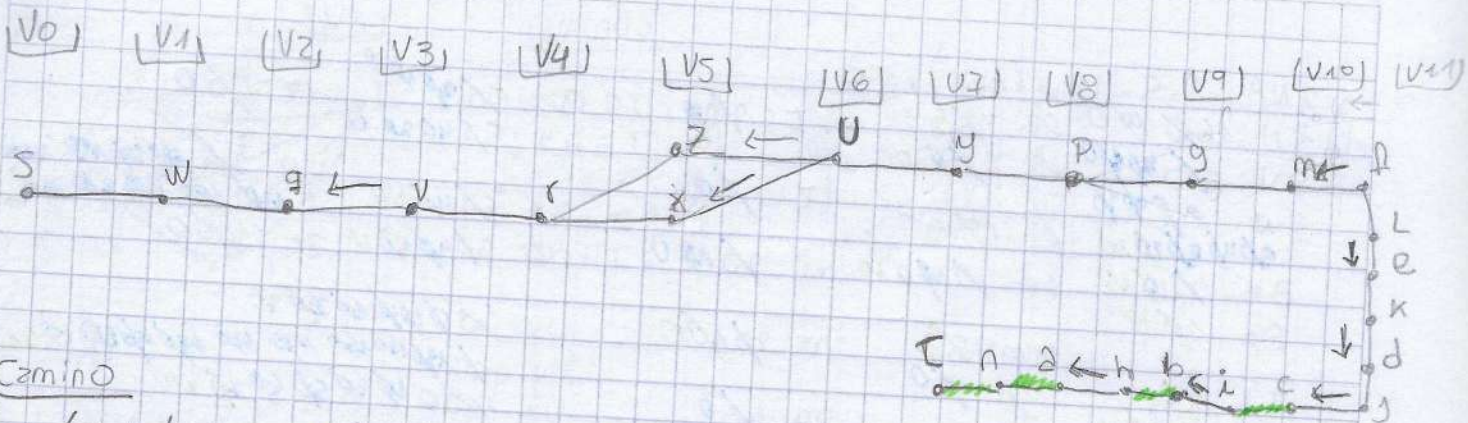
check: $cap(s) = cap(s, \bar{s}) = \sum [x \in s] [y \in \bar{s}] [xy \in E] c(xy)$

$$= c(\vec{BF}) + c(\vec{BD}) + c(\vec{BH}) + c(\vec{CF}) + c(\vec{CH}) + c(\vec{EI}) + c(\vec{EH}) + c(\vec{EJ}) + c(\vec{IT}) + c(\vec{SA})$$

$$= 14 + 9 + 1 + 4 + 20 + 16 + 2 + 7 + 15 + 20$$

Luego mi corte es minimal y $f(fmax)$ mal y $cap(s) = 108 = v(fmax)$ termino check esta todo OK!!

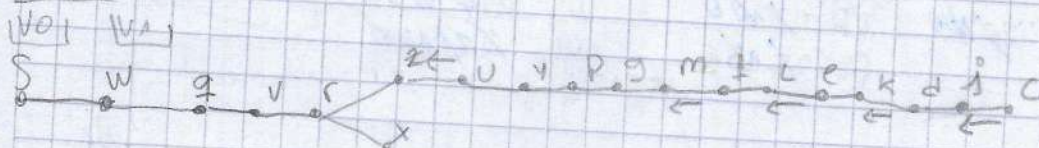
3 NA



Camino

$S \xrightarrow{10} W \xrightarrow{10} g \xrightarrow{10} v \xrightarrow{10} r \xrightarrow{10} z \xrightarrow{10} U \xrightarrow{10} y \xrightarrow{10} P \xrightarrow{10} g \xrightarrow{10} m \xrightarrow{10} f \xrightarrow{10} L \xrightarrow{10} e \xrightarrow{10} k \xrightarrow{10} d \xrightarrow{10} j \xrightarrow{10} c \xrightarrow{10} T$

4 NA



No puedo llegar a T el algoritmo termina

Corte minimal $S = \{S, W, g, v, r, z, x, U, y, P, g, m, f, L, e, k, d, j, c\}$

$$\begin{aligned} \text{Check: } \text{cap}(S) &= \text{cap}(S, \bar{S}) = \sum_{[x \in S][y \in \bar{S}][x, y \in E]} c(x, y) \\ &= \text{cap}(z, \bar{S}) + \text{cap}(x, \bar{S}) + \text{cap}(p, \bar{S}) + \text{cap}(g, \bar{S}) + \text{cap}(m, \bar{S}) + \text{cap}(f, \bar{S}) \\ &\quad + \text{cap}(k, \bar{S}) + \text{cap}(j, \bar{S}) + \text{cap}(c, \bar{S}) \\ &= 100 + 60 + 20 + 20 + 10 + 20 + 20 + 20 + 20 + 10 \\ &= \boxed{300} \end{aligned}$$

$$\begin{aligned} V(f_{\max}) &= \text{out}_f(S) = 10 + 10 + 10 + 10 + 20 + 20 + 10 + 10 + 20 + 20 + 100 + 60 \\ &= \boxed{300} \end{aligned}$$

Luego corte minimal y f flujo maximal y $\text{cap}(S) = 300 = V(f_{\max})$ termino
check está todo ok!!

(3) Modificar ligeramente el algoritmo de Dinic para que en networks en donde todas las capacidades son 1 corra en tiempo $O(nm)$ (i.e. halle un blocking flow en tiempo $O(m)$), probando esta cota (cambiar el avanzar y retroceder inteligentemente para poder eliminar e incrementar flujo).

Pseudo código original

Procedure Dinic (input: NA, output: bloqueos en NA)
 $g = 0$

FLAG = 1

WHILE (FLAG)

$P := [s]$; $X = s$

WHILE (($x \neq t$) AND FLAG)

IF ($r^+(x) \neq \emptyset$)

AVANZAR (Tomar $y \in r^+(x)$
 Agregar "y" a P, hacer $X = y$)

ELSE ($r^+(x) = \emptyset$)

IF ($X \neq s$) retroceder

hacer el lado px a 0 y volver a intentar

Tomar $z = \text{elem. ant. a } x \text{ en } P$

Borrar X de P

Borrar el lado zx del NA

Hacer $x = z$ (nuevo pivote p/otros)

ELSE

FLAG = 0

FI

Hay camino aumentante entre s y t

IF ($X = t$) then

Aumentar flujo a lo largo de P

Borrar lado st usado

FI

return (g)

Pseudo código modificado

$g = 0$

STOPFLAG = 1 // para saber cuando parar

WHILE (STOPFLAG)

camino $P := [s]$, $X = s$ // inicialización de x y del camino P

WHILE ($X \neq t$)

IF ($r^+(x) \neq \emptyset$) THEN

• AVANZAR (x), ($x \neq s$)

ELSE IF ($r^+(x) \neq \emptyset$) THEN IF ($x \neq s$)

RETROCEDER (x)

ELSE

STOPFLAG = 0

FI

od

od

RETURN (g)

• Tomar $y \in r^+(x)$,

agregar y a P ,

aumentar flujo a lo largo de x ; AVANZAR (x)

Borrar lado xy en NA;

hacer $x = y$;

• Tomar $z = \text{elem. ant. a } x \text{ en } P$

Borrar x de P

Borrar el lado zx de NA

Hacer $x = z$

RETROCEDER (x)

Proceder de los elementos.

retroceder del NA $i--$; Fin retroceder

$E = \min\{c(P[i]) - c(P[i-1])\}$ para $i=2$ a n

for $i=2$ a n $g += E$

Fin incrementar

Papel de la

Even ^{forma} "guardar" esa info es simplemente borrar x , o bien, si hacemos backtrack desde x a z , borrar el lado $\overrightarrow{xz}$

Diferencia entre Dinic y Dinica even es en cómo y cuándo se actualiza el network a medida que encontramos nuevos caminos

- Ambos borran lados saturados en un camino pero even borra varios lados

(o vértices) extra mientras corre DFS, c/ vez que tiene que hacer un backtrack.

Versión original de Dinitz, el network auxiliar se construye de forma tal que DFS nunca tenga que hacer backtrack.

y c/ vez que se encuentra un camino entre s y t y se cambia el flujo también se cambia el network auxiliar para seguir teniendo esta propiedad

Even, no se pierde tanto tiempo actualizando el Network auxiliar entre caminos, pero las búsquedas DFS no demoran todas iguales pues algunas modificarán el network más que otras.

Even es más "lazy" y sólo borra lados si los encuentra y se da cuenta que no los necesita, Dinitz borra todos los lados que sabe que son inútiles aún si luego nunca los encontraría.

3dmic

Red dental

Idea usar modificación similar a la de Dinic-even, sólo que además de eliminar un lado del network auxiliar al hacer backtrack (esto era que el lado estaba saturado) Podemos eliminar el lado una vez que sabemos que el flujo que pasa por el mismo es 1 ya que sabemos que la capacidad de todos los lados es 1 por enunciado del ejercicio. y también podemos ahorrarnos el paso de incrementar el flujo ya que sabemos que el valor del flujo que pasa por un camino saturado es 1, podemos asignar el flujo a c/lado mientras aumentamos el camino.

Modificaciones en Avanzar y Retroceder:

- Avanzar(x): Elegimos algún y de $r^+(x)$, agregamos y a la pila que contiene el camino, cambiamos $x=y$, eliminamos $\vec{xy}$ del network auxiliar y definimos $g(\vec{xy})=1$, donde g es el flujo
- Retroceder(x): Tomamos z el vértice anterior a x en la pila, hacemos $x=z$ y $g(\vec{zx})=0$

Avanzar y retroceder son $O(1)$. Además sólo se va a avanzar una vez sobre un lado ya que al avanzar se elimina dicho lado, debido a esto también se retrocede sólo una vez sobre un lado, máximo pueden haber m palabras si todas son de la forma AR . Si m son la cnt de palabras posibles y n la cnt de letras en la misma

$$O(n \cdot m) = O(m)$$

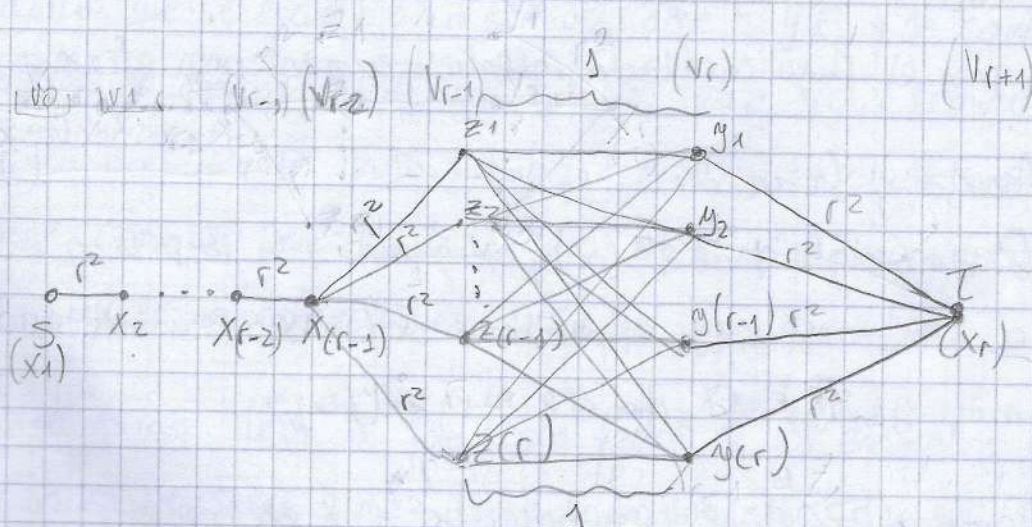
↳ Blocking Flow en tiempo $O(m)$

avanzamos si $r^+(x)$ no es vacío
retrocedemos si $x \neq s$

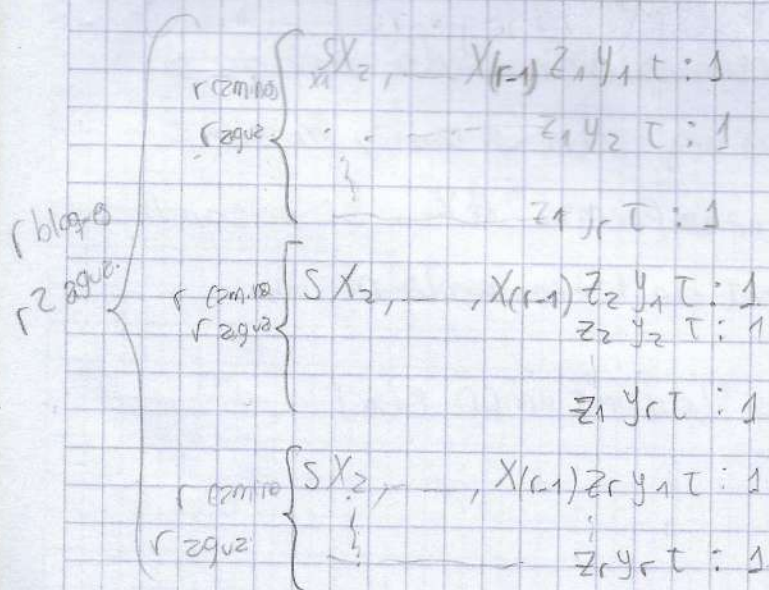
2) vimos que la complejidad de Dinic para encontrar un blocking flow es $O(nm) = O(n^3)$ en networks densos. Probar que es $\Omega(n^3)$ usando los siguientes networks para $r \geq 2$: $n = 3r$ vértices

$X_i, Z_i, Y_i \quad i = 1, \dots, r$ y $X_1 = s$ $X_r = t$
primero ultimo

- los lados son $X_i X_{i+1}$ todos con capacidad r^2 , para todo $i < r-1$
- lados $X_{r-1} Z_j$ de cap. $r^2 \quad \forall j$
- lados $Z_j Y_k$ de cap 1 $\forall j, k$
- lados $Y_k t$ de cap. $r^2 \quad \forall k$.



Nota que para este ejemplo podemos tomar el network original como el network original porque el caso Dinic queda igual.



$\Rightarrow r^2$ caminos

para $j, k = 1, \dots, r$ Tenemos r^2 caminos dados $s, x_2, \dots, x_{r-1}, z_j, y_k, t$:

Recorrer un camino con greedy es $O(r+2) = O(r) = O(\frac{n}{3}) = O(n)$

Encontrar un blocking flow = complejidad (encontrar un camino con greedy)

$O(\# \text{ cant de camino})$

$= O(n) \cdot O(r^2)$

$= O(n) \cdot O\left(\left(\frac{n}{3}\right)^2\right) = O(n) \cdot O(n^2) = \boxed{O(n^3)}$

$\frac{n-3r}{3} = r$

$\therefore O(n^3)$ es la complejidad de Dinic para este ejemplo

Corrección ej. 3: Algoritmo Dinic modificarlo para que en ^{network} donde todas las cap. son 1 corra en tiempo $O(nm)$ (i.e. hallar un blocking flow en tiempo $O(m)$). Probando esta cota cambiar el AVANZAR y RETROCEDER inteligentemente para poder eliminar/incrementar flujo

$g = 0$

FLAG = 1

WHILE (FLAG)

$P := [s]$; $X = s$ // inicialización de X y del camino P.
no llegue al final

WHILE ($X \neq t$) AND FLAG)

IF ($\Gamma^+(x) \neq \emptyset$) THEN

Tomar $y \in \Gamma^+(x)$;
Agregar y a P ;
Aumentar Flujo a la arista $\overrightarrow{xy}$
Borrar lado $\overrightarrow{xy}$ en NA ;
Hacer $X = y$;

Avanzar(x)

ELSE IF ($\Gamma^+(x) = \emptyset$)

IF ($X \neq s$)

Tomar $z = \text{Elem. ant. a } x \text{ en } P$;
Borrar X de P ;
Borrar el lado $\overrightarrow{zy}$ de NA ;
Hacer $X = z$;

Retroceder(x)

ELSE FLAG = 0

Blocking flow

Papel para complejidad total de algoritmo

Return (g)

La idea es usar una modificación a la de Dinic-occidental,
sólo que además de eliminar un lado del network auxiliar al hacer
Back (esto era que estaba saturado) acá podemos eliminar el lado una
vez que sabemos que el flujo que pasa por el mismo es 1 ya que
sabemos por enunciado del ejercicio que la capacidad de todos los lados
es 1. y además eliminamos el incrementar el flujo ya que sabemos
que el valor del flujo que pasa por un camino saturado es 1 y
podemos asignar el flujo a cada lado mientras aumentamos el camino.

Modificamos Avanzar y retroceder.

- Avanzar(x) = Elegimos algún y de $\Gamma^+(x)$, agregamos y a la pila que
contiene el camino, cambiamos $x=y$, eliminamos $\vec{x\bar{y}}$ del network
auxiliar y definimos $g(\vec{x\bar{y}})=1$ donde g es el flujo.

- Retroceder(x): Tomamos z el vértice anterior a x en la pila,
hacemos $x=z$ y $g(\vec{zx})=0$ con g es el flujo

Sabemos que cada Avanzar y retroceder es $O(1)$

→ Ahora eliminamos I que era Recorrer un camino de long n dos veces $O(n)$.

Además el while que tenemos en nuestro pseudocódigo por cada
vez que se recorre es $O(1)$ por operaciones que hace el avanzar y
retroceder pero esto se realiza hasta que termina de eliminar
todos los lados. y la cantidad de lados son m , por lo menos.
eliminamos 1 lado cuando avanza. y cuando retroceder eliminamos
el lado y el vértice.

- entonces como siempre elimino 1 lado y la cant son m .
entonces la complejidad del algoritmo es $O(m)$